

Lab(Optional B: JavaScript): Creating a Swagger Documentation for REST API



Estimated Time: 45 minutes

In this lab, you will understand how to create a Swagger documentation for your REST APIs using Node.js and Express.

Learning objectives

After completing this lab, you will be able to:

- Use the Swagger Editor to create Swagger documentation for REST API
- Use SwaggerUI to access the REST API endpoints of an application
- Generate code with the Swagger documentation

Prerequisites

- You must be familiar with Docker applications and commands
- You must have a good understanding of REST API.
- Knowledge of JavaScript and Node.js is highly recommended

Task 1 - Getting your application started

1. Open a terminal window by using the top menu in the IDE: **Terminal** > **New Terminal**, if you don't have one open already.

2. In the terminal, clone the repository that has the Swagger documentation and the REST API code ready by pasting the following command. The repository that you clone has code that will run a REST API application, which can be used to organize tasks.

```
git clone https://github.com/lin-developer/skills-network/ibagq-crud_microservices_js.git
```

3. Change the working directory to **ibagq-crud_microservices_js/swagger_example** by running the following command.

```
cd ibagq-crud_microservices_js/swagger_example
```

4. Run the following commands to install the required packages.

```
npm install express cors swagger-ui-express
```

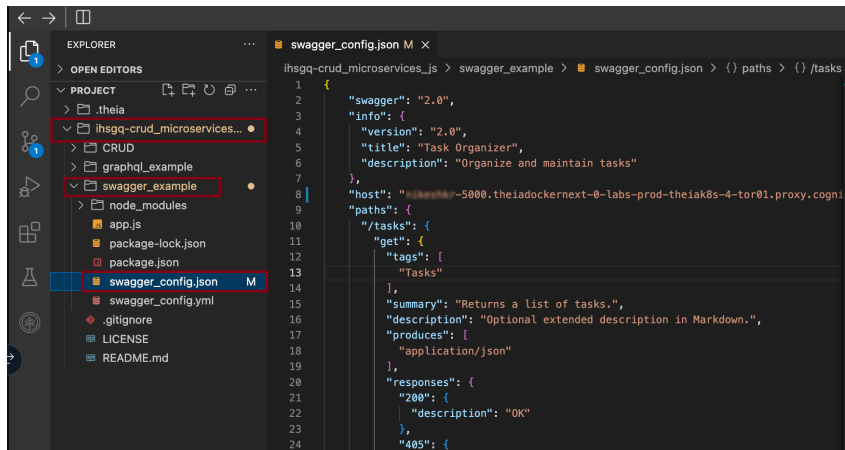
5. Now start the application which serves the REST API on port number 5000.

```
node app.js
```

6. Click on the Skills Network button on the left, it will open the "Skills Network Toolbox." Then, click Launch Application; from there, you enter the port no. as 5000 and click Your Application button. This will open a new browser page, which accesses the application you just ran.

7. Copy the URL on the address bar.

8. From the file menu, go to **ibagq-crud_microservices_js/swagger_example/swagger_config.json** to view the file on the file editor:



9. In the file editor, paste the application URL that you copied where it says **your application URL** without the protocol (https://) and do not put a "/" at the end of the URL and save the file.

10. Copy the entire content of the file **swagger_config.json**. You will need this copied content to generate SwaggerUI.

11. Click on this link <https://editor.swagger.io/> to go to the Swagger Editor.

12. From the File menu, click on Clear Editor to clear the content of the Swagger Editor.

13. Paste the content you copied from **swagger_config.json** on the left side. You will get a prompt that says **would you like to convert your JSON into YAML?**. Press **Cancel** to paste the content.

14. You will see that the UI is automatically populated on the right.

Swagger Editor

File Edit Generate Server Generate Client About

Try our new Editor

```
1 {
2   "swagger": "2.0",
3   "info": {
4     "version": "2.0",
5     "title": "Task Organizer",
6     "description": "Organize and maintain tasks"
7   },
8   "host": "5000.theiadocker-1-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai",
9   "paths": {
10     "/tasks": {
11       "get": {
12         "tags": [
13           "Tasks"
14         ],
15         "summary": "Returns a list of tasks.",
16         "description": "Optional extended description in Markdown.",
17         "produces": [
18           "application/json"
19         ],
20         "responses": {
21           "200": {
22             "description": "OK"
23           },
24           "405": {
25             "description": "Invalid Input"
26           }
27         }
28       },
29       "/task/{taskname}": {
30         "get": {
31           "tags": [
32             "Task specific activity"
33           ],
34           "summary": "Returns a task by name.",
35           "parameters": [
36
```

Task Organizer^{2.0}

[Base URL: lavanyas-5000.theiadocker-1-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai]

Organize and maintain tasks

Tasks

GET

/tasks Returns a list of tasks.

POST

/task Add a task

Task specific activity

GET

/task/{taskname} Returns a task by name.

DELETE

/task/{taskname} Delete a task by name.

15. Now you can test each of the endpoints. Four tasks have been already added for you, when the application was started. Click on the down arrow next to GET **tasks**.

GET

/tasks Returns a list of tasks.

16. Click **Try it out**. This will allow you to try your REST API endpoint.

GET

/tasks Returns a list of tasks.

Optional extended description in Markdown.

Parameters

Try it out

No parameters

Responses

Response content type

application/json

Code	Description
200	OK
405	Invalid Input

17. Click **Execute** to invoke a call to your REST API. This is a GET request that does not take any parameters. It returns the task as an **application/json**.

GET

/tasks Returns a list of tasks.

⤴

Optional extended description in Markdown.

Parameters

Cancel

No parameters

Execute

Responses

Response content type application/json

⌵

Code	Description
200	OK
405	Invalid Input

18. You can scroll down to view the output of the API call.

Curl

```
curl -X 'GET' \
  'https://lavanyas-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/tasks'
-H 'accept: application/json'
```

Request URL

```
https://lavanyas-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/tasks
```

Server response

Code	Details
200	Response body <pre>{ "tasks": [{ "description": "Do the laundry this weekend", "name": "Laundry" }, { "description": "Finish assignment by Friday", "name": "Assignment" }, { "description": "Call family Sunday morning", "name": "Call family" }, { "description": "Pay the electricity and water bill", "name": "Pay bills" }] }</pre> ⌵ Download

19. Try to do the following:

- Add a task
- Retrieve the tasks to see if your task is added to the list
- Get the details on one task
- Delete a task and check the list to verify that it is deleted.

20. From the File menu, click **Clear Editor** to clear the content of the Swagger Editor.

Swagger Editor

File Edit Generate Server Generate Client About

Supported by SMARTBEAR

```
1 {
2   "swagger": "2.0",
3   "info": {
4     "version": "1.0",
5     "title": "T
6     "description": "contain tasks"
7   },
8   "host": "lav
9   "paths": {
10    "/tasks": {
11      "get": {
12        "tags": [
```

Import URL

Import file

Save as JSON

Convert and save as YAML

Clear editor

Task 2 - Creating Swagger documentation and generating server code

1. Now you will create a REST API with Swagger documentation. To start with, let's define your application.

- It will adhere to Swagger 2.0 version.
- This is the first version of the application.
- It will have one endpoint /greetings, which returns the list of greetings as a JSON object.

2. Copy and paste the following JSON in the Swagger Editor. You will get a prompt that says you'd like to convert your JSON into YAML. Press **Cancel** to paste the content.

```
{
  "swagger": "2.0",
  "info": {
    "version": "1.0",
    "title": "Our first generated REST API",
    "description": ">2>This is a sample server code the is generated from Swagger Documentation with Swagger Editor>2>"
  },
  "paths": {
    "/greetings": {
      "get": {
        "summary": "Returns a list of Greetings",
        "tags": ["Hello in different languages"],
        "description": "Returns greetings in different languages",
        "produces": [
          "application/json"
        ],
        "responses": {
          "200": {
            "description": "OK"
          }
        }
      }
    }
  }
}
```

Swagger Editor
Supported by SMARTBEAR

File Edit Generate Server Generate Client About

Try our new Editor

```
1 {
2   "swagger": "2.0",
3   "info": {
4     "version": "1.0",
5     "title": "Our first generated REST API",
6     "description": "This is a sample server code the is generated from
7       Swagger Documentation with Swagger Editor"
8   },
9   "paths": {
10    "/greetings": {
11      "get": {
12        "summary": "Returns a list of Greetings",
13        "tags": ["Hello in Different Languages"],
14        "description": "Returns greetings in different languages",
15        "produces": [
16          "application/json"
17        ],
18        "responses": {
19          "200": {
20            "description": "OK"
21          }
22        }
23      }
24    }
25  }
26 }
```

Our first generated REST API ^{1.0}

This is a sample server code the is generated from Swagger Documentation with Swagger Editor

Hello in Different Languages

GET /greetings Returns a list of Greetings

You will see the Swagger UI automatically appearing on the right. You cannot test it yet as your application is not defined and running yet.

3. From the menu on top, click **Generate Server** and select **nodejs-server**. This will automatically generate the server code as a zip file named **nodejs-server-generated.zip**. Download the zip file to your system.

Swagger Editor
Supported by SMARTBEAR

File Edit Generate Server Generate Client About

Try our new Editor

```
1 {
2   "swagger": "2.0",
3   "info": {
4     "version": "1.0",
5     "title": "Our first generated REST API",
6     "description": "This is a sample server code the is generated from
7       Swagger Documentation with Swagger Editor"
8   },
9   "paths": {
10    "/greetings": {
11      "get": {
12        "summary": "Returns a list of Greetings",
13        "tags": ["Hello in Different Languages"],
14        "description": "Returns greetings in different languages",
15        "produces": [
16          "application/json"
17        ],
18        "responses": {
19          "200": {
20            "description": "OK"
21          }
22        }
23      }
24    }
25  }
26 }
```

ada-server	jaxrs-resteasy	restbed
aspnetcore	jaxrs-resteasy-eap	rust-server
erlang-server	jaxrs-spec	scala-lagom-server
finch	kotlin-server	scalatra
go-server	lumen	sinatra
haskell	msf4j	slim
inflector	nancyfx	spring
java-pkms	nodejs-server	undertow
java-play-framework	php-silex	ze-ph
java-vertx	php-symfony	
jaxrs	pistache-server	
jaxrs-cxf	python-flask	
jaxrs-cxf-cdi	rails5	

Our first generated REST API ^{1.0}

This is a sample server code the is generated from Swagger Documentation with Swagger Editor

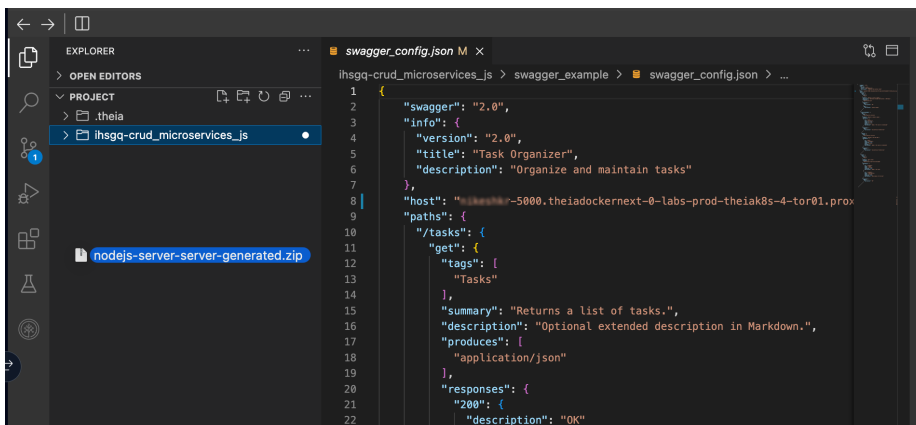
Hello in Different Languages

GET /greetings Returns a list of Greetings

Note: If you are unable to download the ZIP file from Swagger or encounter errors while downloading, you can use the following command to download a working file directly:

```
wget -O nodejs-server-generated.zip "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/1970214891-MSGS043-whojs-server-generated.zip"
```

4. In your lab environment, click the **PROJECT** folder and drag and drop the zip file there.



5. On the terminal, go to the `/home/project` directory.

```
cd /home/project
```

6. Check to see if the zip file that you just dragged and dropped exists.

```
ls nodejs-server-server-generated.zip
```

7. Unzip the contents of the zip file into a directory named `nodejs-server-generated` by running the following command.

```
unzip nodejs-server-server-generated.zip -d nodejs-server-generated/
```

8. Change to the `nodejs-server` folder inside the folder you just extracted the zip file into.

```
cd nodejs-server-generated/nodejs-server-server
```

9. The entire server setup along with endpoint is done for you already. Let's install necessary dependencies.

```
npm install
```

10. Run the below commands to start node server. The server generated code automatically is configured to run on port 8080.

```
node index.js
```

This confirm that the node server is running.

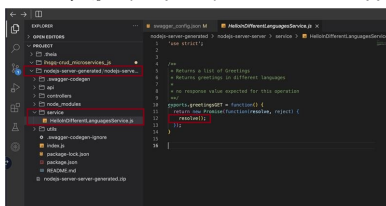
```

theia@theiadocker-0-labs-prod-theiak8s-4-tor01:~/home/project/nodejs-server-generated/nodejs-server-server$ node index.js
Your server is listening on port 8080 (http://localhost:8080)
Swagger-ui is available on http://localhost:8080/docs

```

11. Now you should stop the server by pressing `ctrl + c`.

12. In the file explorer, go to `nodejs-server-generated/nodejs-server-server/service/helloDifferentLanguagesService.js`. This is where you need to implement your actual response for the REST API.



13. Replace `resolve()` with the following code.

```

let hellos = {
  "English": "hello",
  "Hindi": "namaste",
  "Spanish": "hola",
  "French": "bonjour",
  "German": "guten tag",
  "Italian": "buona",
  "Chinese": "nǐ hǎo",
  "Portuguese": "olá",
  "Arabic": "salam alaikum",
  "Japanese": "konnichiwa",
  "Korean": "안녕하세요",
  "Russian": "доброе утро"
}

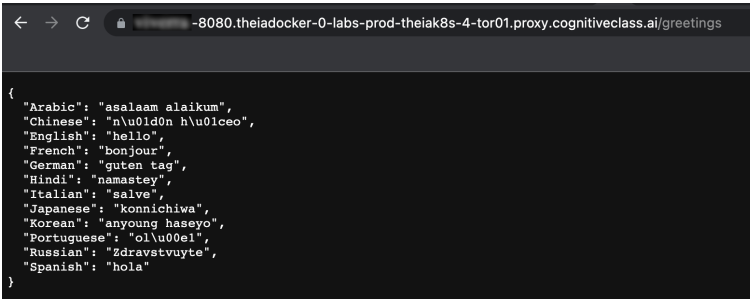
resolve(hellos);

```

14. Run the node server again.

```
node index.js
```

15. Click the Skills Network button on the left, it will open the "Skills Network Toolbox." Then click Launch Application, from there you enter the port no. as 8080 and click Your Application button. This will open a browser window. Append the path /greetings to the URL. You should see the greetings in the page.



The screenshot shows a web browser window with the address bar displaying the URL: -8080.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/greetings. The main content area of the browser displays a JSON object containing greetings in multiple languages.

```
{
  "Arabic": "assalaam alaikum",
  "Chinese": "n\u00f0d\u00f0n h\u00f0l\u00f0ce\u00f0",
  "English": "hello",
  "French": "bonjour",
  "German": "guten tag",
  "Hindi": "namastey",
  "Italian": "salve",
  "Japanese": "konnichiwa",
  "Korean": "anyoung haseyo",
  "Portuguese": "ol\u00f0e!",
  "Russian": "Zdravstvuyte",
  "Spanish": "hola"
}
```

Congratulations! You have successfully completed the task.

Author:

[Rajashree Patil](#)

[Nikhil Kumar](#)

© IBM Corporation. All rights reserved.